Master 2 Informatique

Data sciences, infrastructure cloud et sécurité

Machine Learning By Practice

Fabrice LAURI

Maître de conférences

à l'Université de Technologie de Belfort-Montbéliard

General Overview

- Introduction
- Data Visualization
- Dimensionality Reduction
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning
- Conclusion

CM #5

Dimensionality Reduction

- Principal Components Analysis
- Kernel PCA
- t-SNE

Unsupervised Learning

- Mean and Median based algorithms
- Mean shift
- DBScan
- Gaussian Mixture Models
- Agglomerative Hierarchical Clustering
- Self-Organizing Map

Fabrice LAURI
Maître de conférences à l'UTBM

Summary of the categories of *Machine Learning*

Fabrice LAURI
Maître de conférences à l'UTBM

Supervised Learning

Models that can predict labels based on labeled training data

Classification

Models that predict labels as two or more discrete categories

Regression

Models that predict continuous labels

Unsupervised Learning

Models that identify structure in unlabeled data

Clustering

Models that detect and identify distinct groups in the data

Dimensionality Reduction

Models that detect and identify lower-dimensional structure in higher-dimensional data

Dimensionality Reduction

Fabrice LAURI
Maître de conférences à l'UTBM

Dimensionality Reduction

Many reasons why dimensionality reduction is useful:

– reduces the curse of dimensionality
– reduces the computational cost
– can remove noise
– can significantly improve the results
– make the dataset easier to work with
– make the results easier to understand
– enable the data to be plotted in 2D or 3D, and make it easier to understand and interpret

Dimensionality Reduction

Three ways to do DR:

– Feature reduction

Deciding whether or not some features are actually useful

– Feature derivation

Deriving new features from the old ones, by applying transforms to the data.

– Clustering

Group together similar datapoints, and see whether fewer features can be used.

Dimensionality Reduction
Principal Components Analysis

PCA is a technique for reducing the number of dimensions in a dataset whilst retaining most information. It is using the correlation between some dimensions and tries to provide a **minimum number of variables** that **keeps the maximum amount of variation** or information about how the original data is distributed.

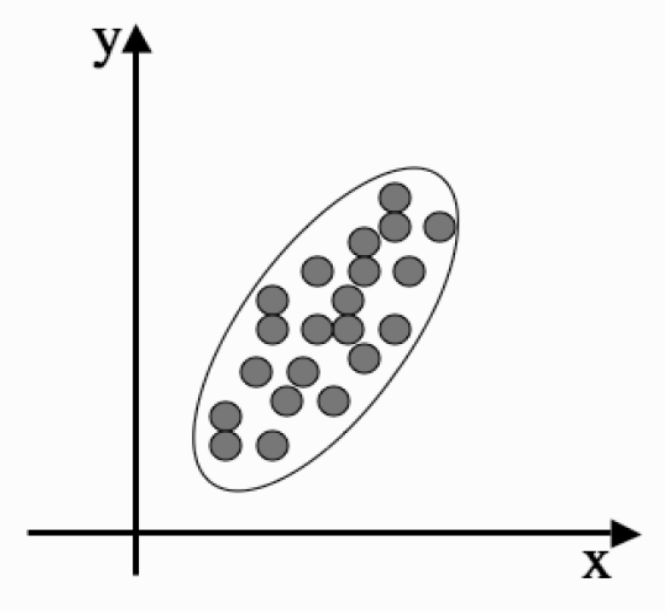
It uses something known as the *eigenvalues* and *eigenvectors* of the data-matrix. These **eigenvectors** of the covariance matrix have the property that they **point along the major directions of variation in the data**. These are the directions of maximum variation in a dataset.

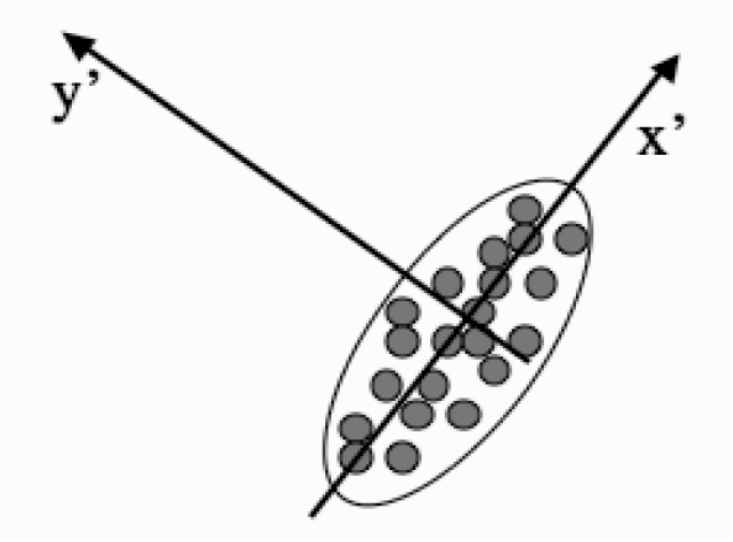
Fabrice LAURI
Maître de conférences à l'UTBM

Dimensionality Reduction
Principal Components Analysis

Principal component=direction in the data with the largest variation.

Eigenvalue=how much stretching we need to do along its corresponding eigenvector.





Fabrice LAURI
Maître de conférences à l'UTBM

Dimensionality Reduction
Principal Components Analysis

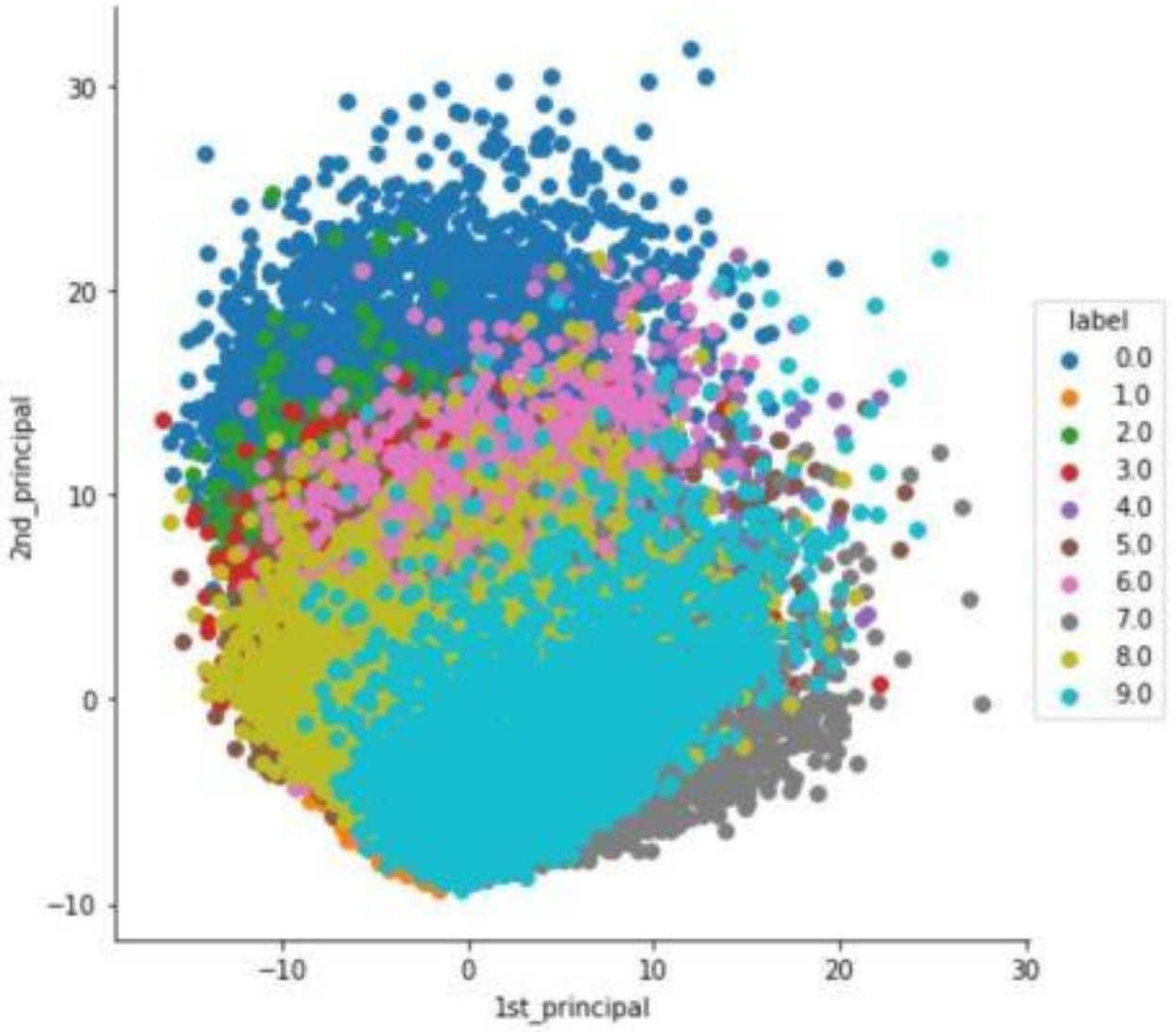
Algorithm:

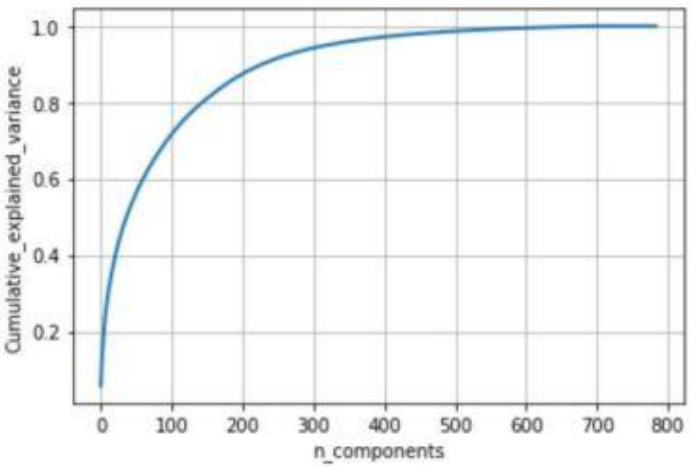
– Write N datapoints x_i as row vectors
– Put these vectors into a matrix X (of size NxM)
– Standardize the data by subtracting off the mean of each column and dividing by the variance, then putting into matrix B
– Compute the covariance matrix $C=1/N\ B^TB$
– Compute the eigenvalues and eigenvectors of C, so $V^{-1}CV=D$
– Sort the columns of D into order of decreasing eigenvalues (same order to the columns of V)
– reject those with eigenvalues less than some threshold, leaving L dimensions in the data

PCA : Code snippet

```
from sklearn.preprocessing import StandardScaler
from scipy.linalg import eigh
import pandas as pd
from sklearn.decomposition import PCA

# Load the Dataset
train = pd.read_csv("data/train.csv")
X_train = train.drop(['label'], axis=1)
y_train = train['label']

standardized_scalar = StandardScaler()
standardized_data = standardized_scalar.fit_transform(X_train)
cov_matrix = np.matmul(standardized_data.T, standardized_data)

lambdas, vectors = eigh(cov_matrix, eigvals=(782, 783))

new_coordinates = np.matmul(vectors, standardized_data.T)
new_coordinates = np.vstack((new_coordinates, y_train)).T

df_new = pd.DataFrame(new_coordinates, columns=["f1", "f2", "labels"])
```

PCA on the MNIST dataset





https://mylearningsinaiml.wordpress.com/2018/09/10/pca-visualization-mnist-data/

Fabrice LAURI
Maître de conférences à l'UTBM

Dimensionality Reduction
t-SNE

t-Distributed Stochastic Neighbor Embedding (t-SNE) is another technique for dimensionality reduction and is particularly well suited for the visualization of high-dimensional datasets.
Contrary to PCA it is not a mathematical technique but a probablistic one.

The original paper describes the working of t-SNE as:

t-Distributed stochastic neighbor embedding (t-SNE) minimizes the divergence between two distributions: a distribution that measures pairwise similarities of the input objects and a distribution that measures pairwise similarities of the corresponding low-dimensional points in the embedding.

Dimensionality Reduction
t-SNE

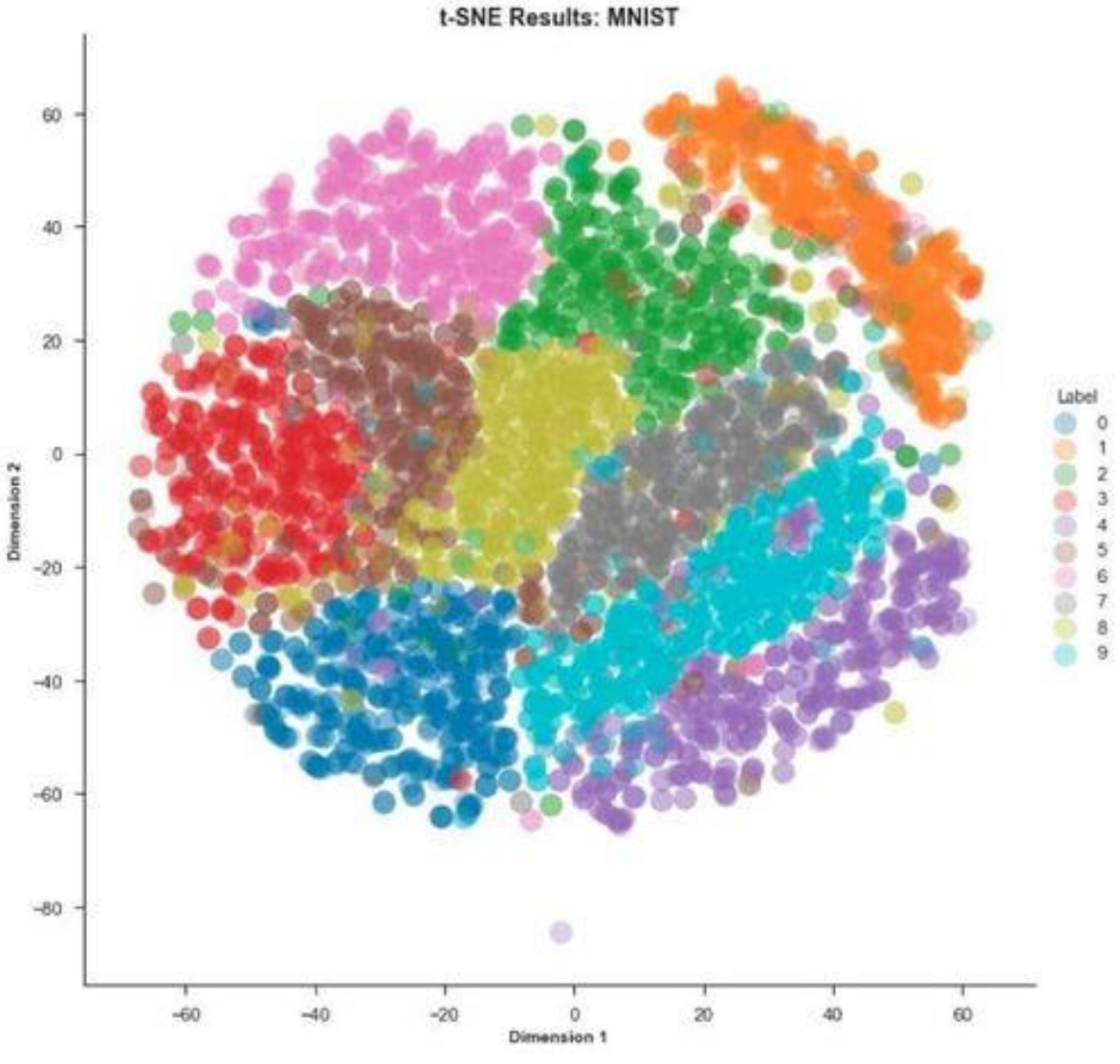
Compared to PCA (Principal Component Analysis), t-SNE is better at capturing non-linear dependencies and provides more meaningful insights and patterns about the different labels.
However, unlike PCA, the cost function of t-SNE is non-convex, which means it can get stuck in local minima, and the compressed dimensions and transformed features are less interpretable.

It is generally recommended to use PCA or TruncatedSVD to reduce the number of dimensions to a reasonable amount (e.g., 50) before applying t-SNE, as this can reduce noise and speed up computations.

t-SNE : Code snippet

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
from keras.datasets import mnist

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Flatten the images
X_train = X_train.reshape((60000, 28 * 28))
X_test = X_test.reshape((10000, 28 * 28))
# Standardize the data
X_train = X_train.astype('float32') / 255
X_test = X_test.astype('float32') / 255
# Apply t-SNE
tsne = TSNE(n_components=2, random_state=0)
tsne_res = tsne.fit_transform(X_train[:1000, :])
# Visualize the result
sns.scatterplot(x=tsne_res[:, 0], y=tsne_res[:, 1], hue=y_train[:1000], palette=sns.hls_palette(10),
legend='full')
plt.show()
```

t-SNE on the MNIST dataset



Fabrice LAURI
Maître de conférences à l'UTBM

Clustering

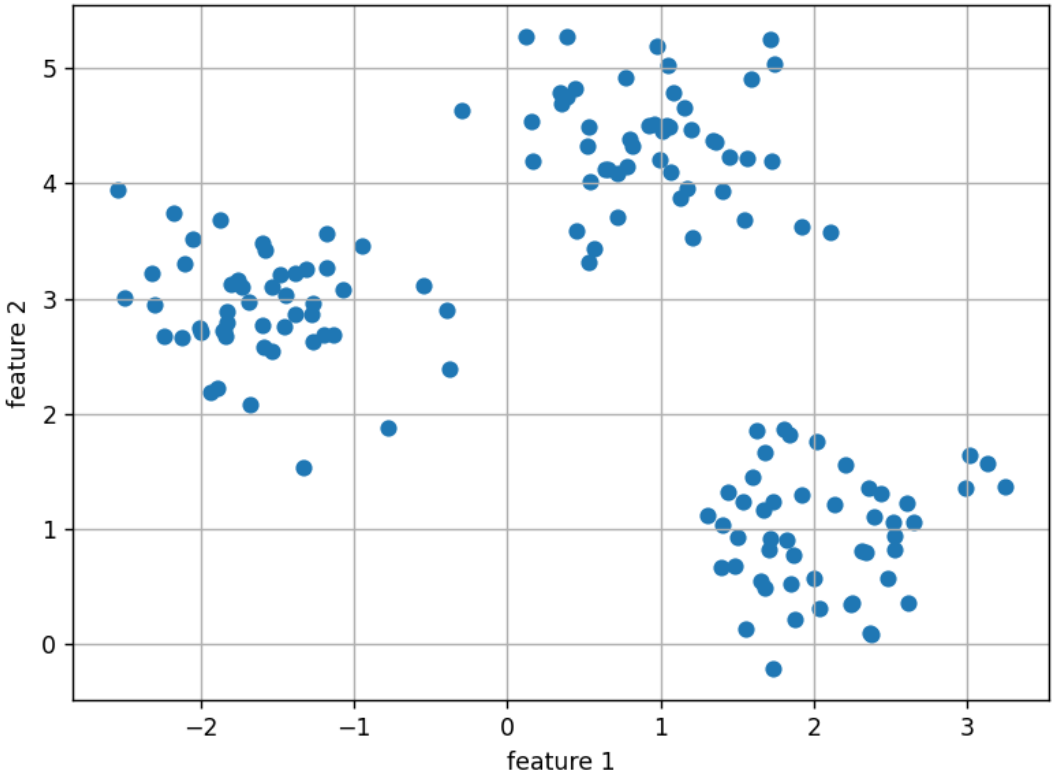
Fabrice LAURI
Maître de conférences à l'UTBM

Unsupervised Learning and clustering

In Supervised Learning, the training set consists of a collection of labelled target data.
In Unsupervised Learning, there is no information about the correct outputs.
Clustering algorithms has to discover the similarities in the data, in order to cluster inputs that are similar together.

Main concepts:
– Input space and weight space
– If two inputs are close together, then it means that their vectors are very similar and so the distance between them is small.
– Inputs that are similar can be clustered together

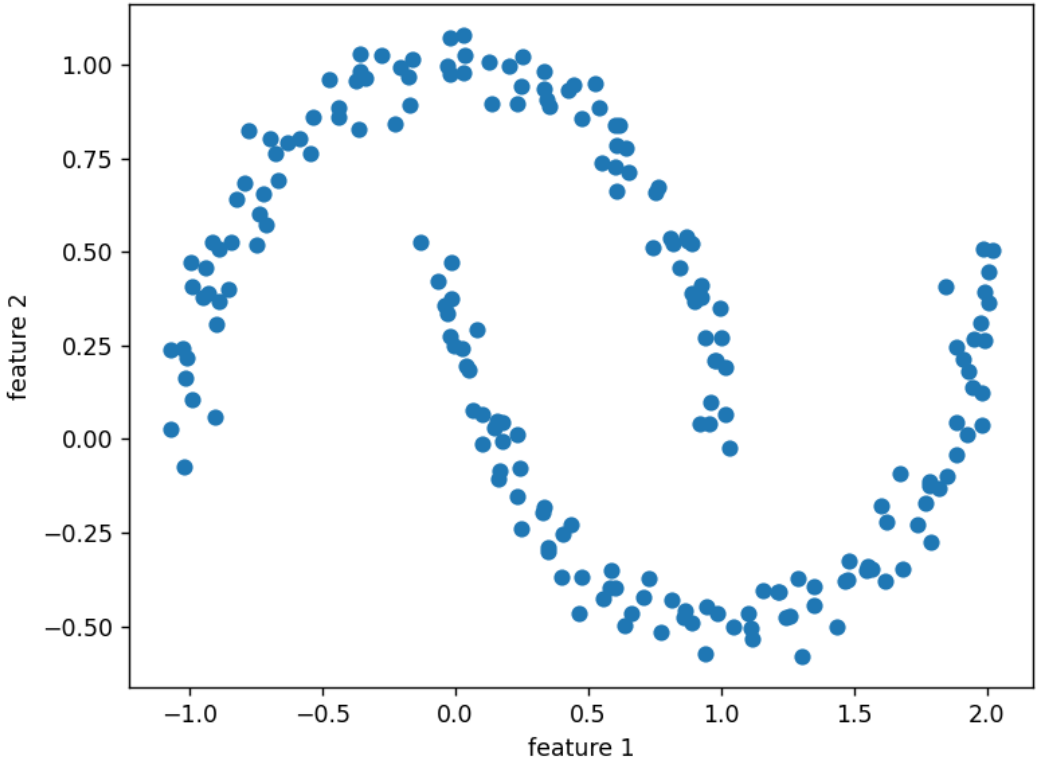
Example of random dataset: Blob dataset




```python
from sklearn.datasets import
make_blobs,make_moons
import matplotlib.pyplot as plt

X,y=make_blobs(
    n_samples=150,
    n_features=2,
    centers=3,
    cluster_std=0.5,
    shuffle=True,
    random_state=0)

plt.scatter(X[:,0],X[:,1])
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.grid()
plt.tight_layout()
plt.show()
```

Fabrice LAURI
Maître de conférences à l'UTBM

Example of random dataset: Moon dataset




```
from sklearn.datasets import
make_blobs,make_moons
import matplotlib.pyplot as plt

X,y=make_moons(
    n_samples=200,
    noise=0.05,
    random_state=0)

plt.scatter(X[:,0],X[:,1])
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.grid()
plt.tight_layout()
plt.show()
```

Fabrice LAURI
Maître de conférences à l'UTBM

Unsupervised Learning
K-Means Algorithm

We want to divide a given input data into K clusters.

Main principles:
– allocate k cluster centers to our input data
– the inputs that are closest to a center k belong to class k

Unsupervised Learning
K-Means Algorithm

Initialisation:

– choose a value for k

– choose k cluster centers randomly or from the datapoints

Learning:

– Repeat

  – For each datapoint x

    – Compute the distance to each cluster

    – Assign the datapoint to the nearest cluster center

  – For each cluster center

    – Move the position of the center to the mean of the points in that cluster

– Until the cluster centers stop moving

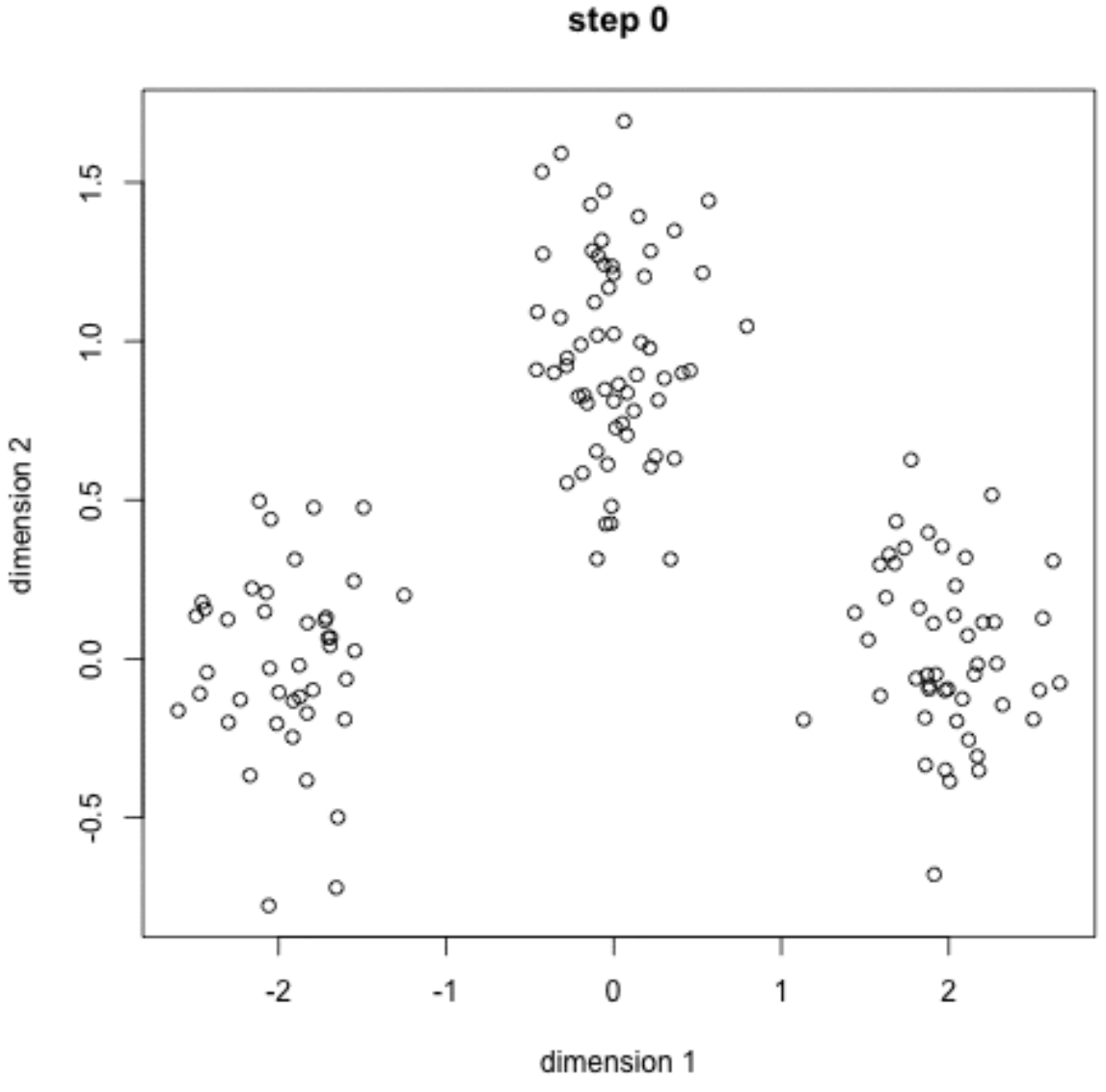
Usage:

– For each test point

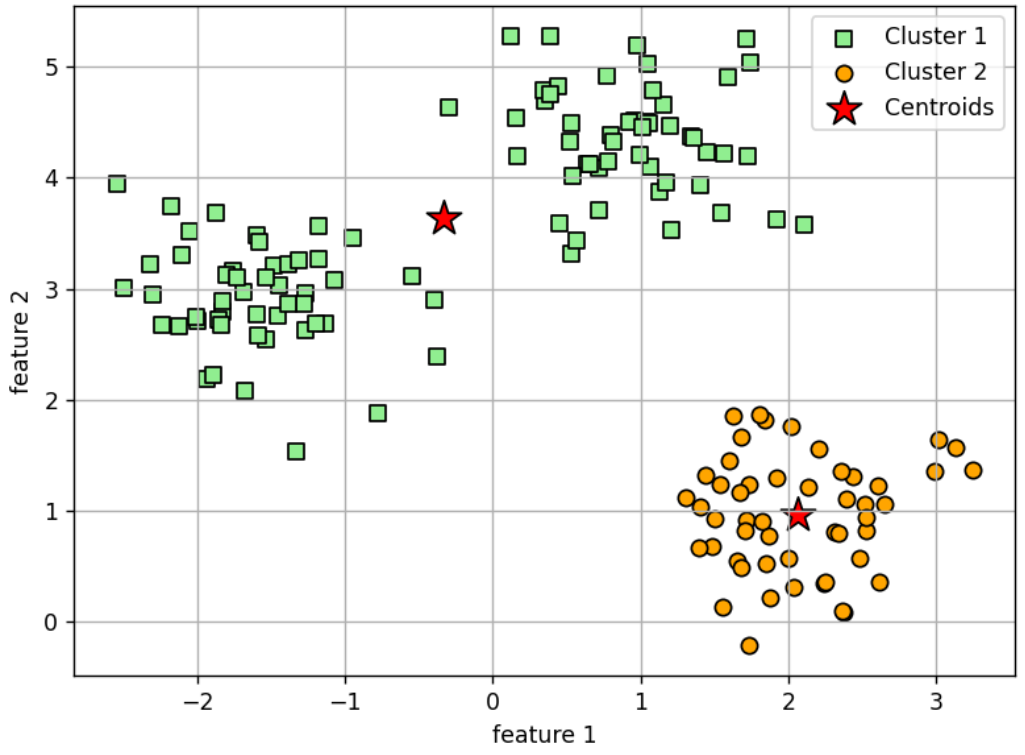
  – Compute the distance to each cluster

  – Assign the datapoint to the nearest cluster center

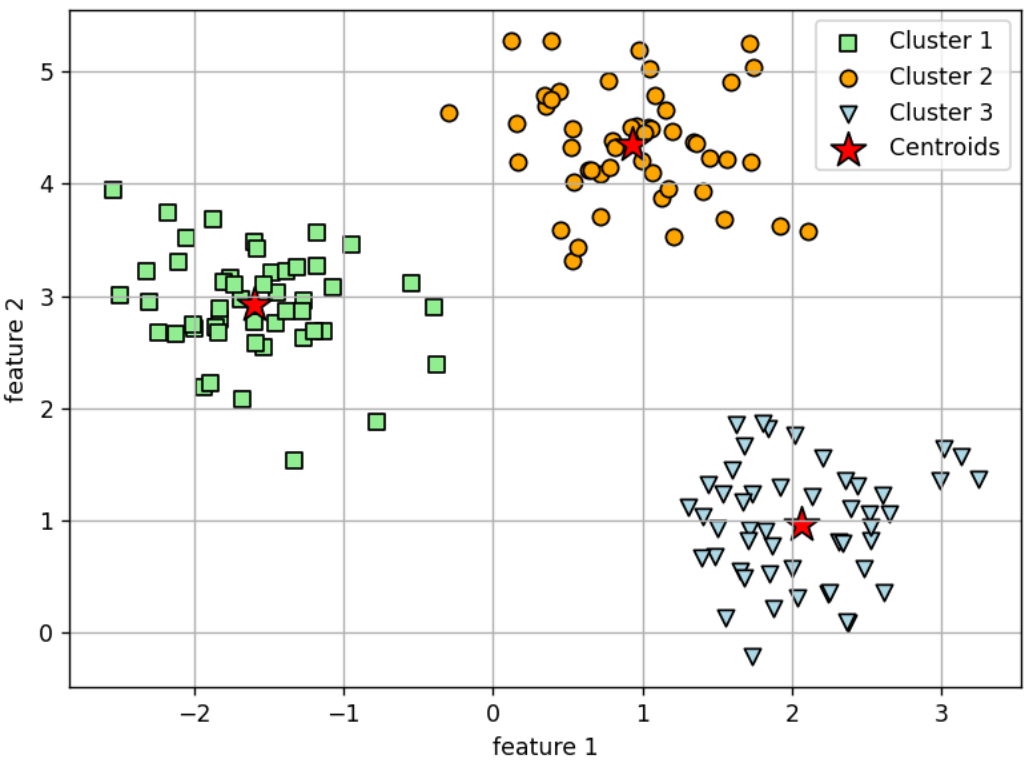
Unsupervised Learning
K-Means Algorithm



K-Means: Code snippet and results



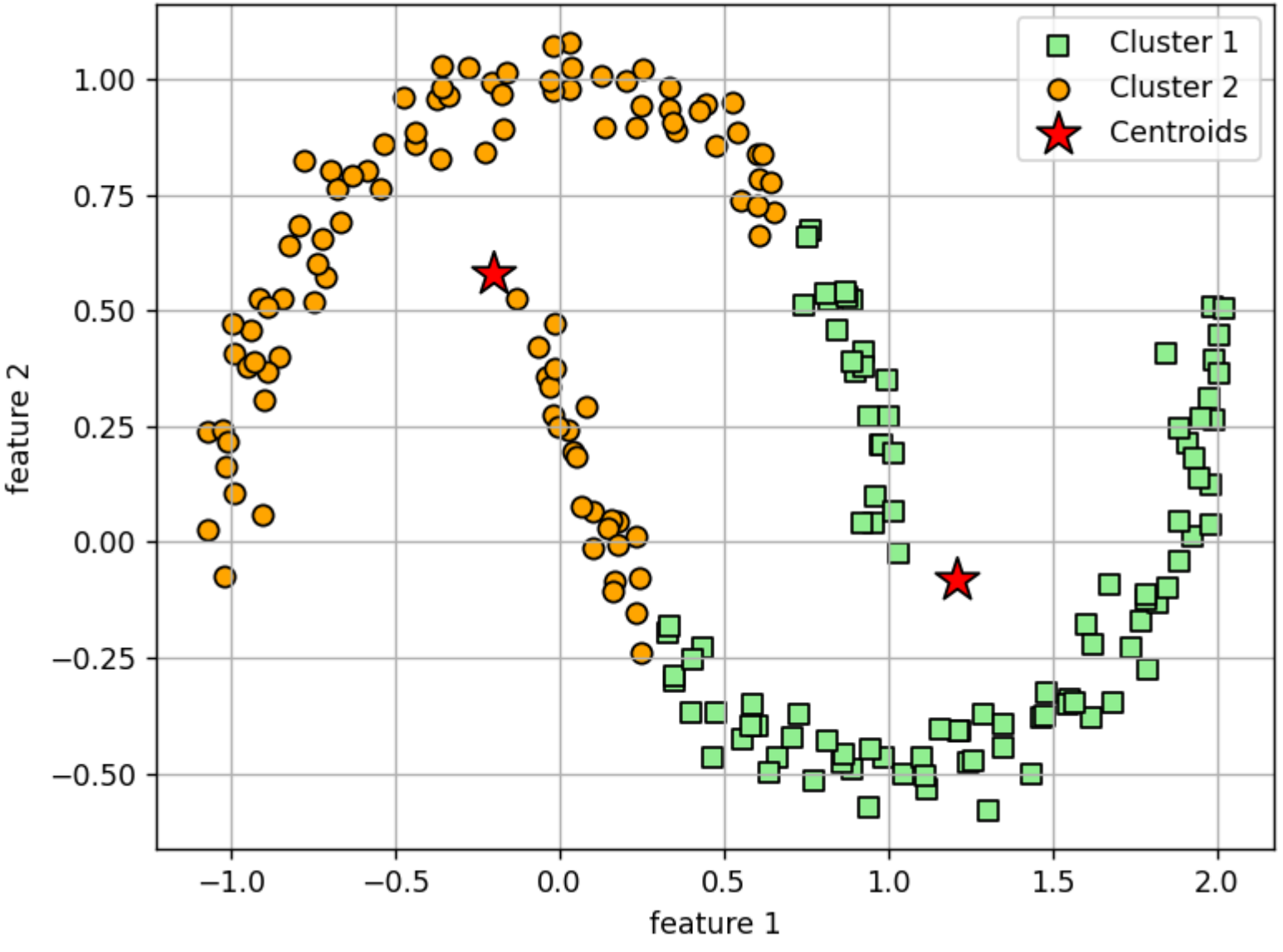

```
from sklearn.cluster import KMeans

km=KMeans(
    n_clusters=2,
    init='random',
    n_init=10,
    max_iter=300,
    tol=1e-4,
    random_state=0)
y_km=km.fit_predict(X)
```




```
from sklearn.cluster import KMeans

km=KMeans(
    n_clusters=3,
    init='random',
    n_init=10,
    max_iter=300,
    tol=1e-4,
    random_state=0)
y_km=km.fit_predict(X)
```

utbm

K-Means: results on Moon dataset



utbm

Unsupervised Learning
K-Means Drawbacks

K-Means assumes we know how many clusters K we should see in the data: not always the case...
And depending upon where the centers are initially positionned, very different solutions can be obtained.

We can get around both of these problems by running the algorithm many different times.

As centers are computed from the mean of datapoints, computing the median can remove noise effectively (less sensitive to outliers).
But it is much slower for larger datasets as sorting is required on each iteration when computing the median vector.

Unsupervised Learning
K-Means++

Choosing the initial centers in K-Means has an direct impact on the performance of the method.

K-Means++ is an initialization algorithm that avoid choosing two initial centers that are too near.

Principal :

• The centers are chosen randomly according to a non-uniform probability distribution function.

• The probaility of choosing a point as an initial center is proportional to the square of its distance to the already chosen centers.

On input, K-Means++ receives the m elements xi to cluster (without the centers). On output, it generates the k initial centers for K-Means.

Fabrice LAURI
Maître de conférences à l'UTBM

Unsupervised Learning
K-Means++ Limits

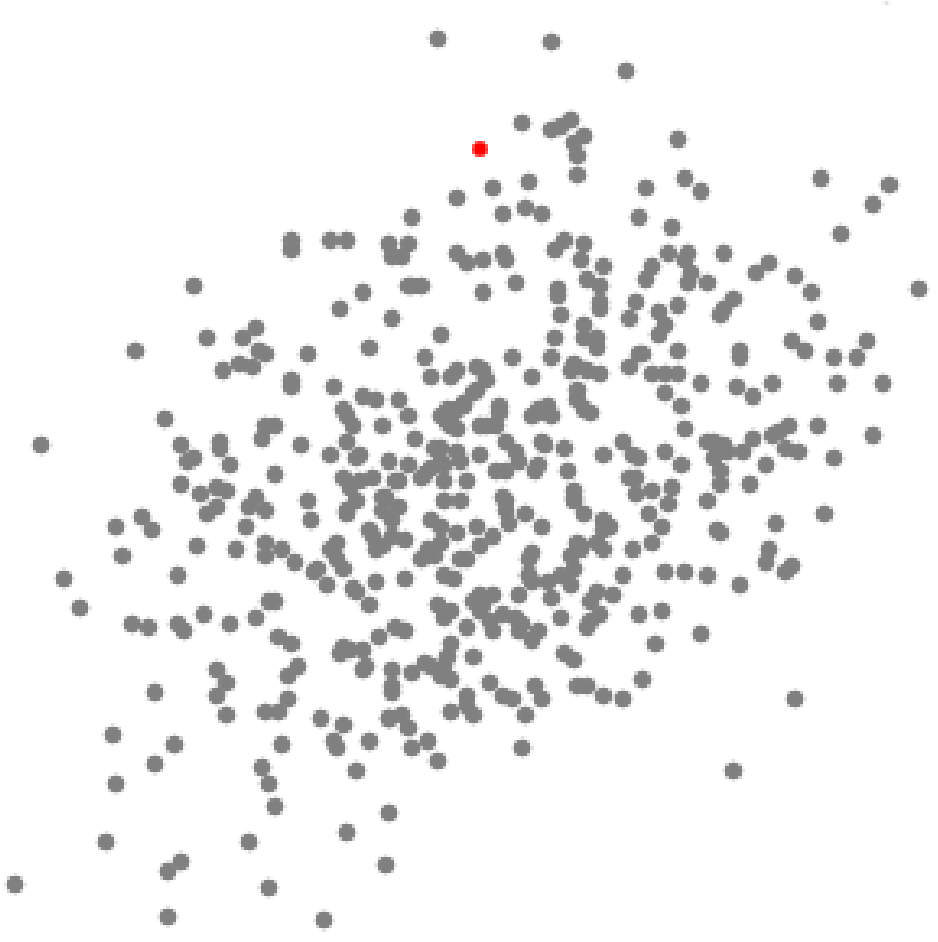
Among the advantages of the method, the K-Means++ has convergence guarantees and can be executed in parallel.

But this technique need finding the closest centers of each element, and this explodes with the number of centers.

Unsupervised Learning
Mean-shift clustering

Mean shift clustering is a sliding-window-based algorithm that attempts to find dense areas of data points.
It is a centroid-based algorithm: the goal is to locate the center points of each group/class, which works by updating candidates for center points to be the mean of the points within the sliding-window. These candidate windows are then filtered in a post-processing stage to eliminate near-duplicates, forming the final set of center points and their corresponding groups.

Unsupervised Learning
Mean-shift clustering



Fabrice LAURI
Maître de conférences à l'UTBM

Unsupervised Learning
Mean-shift clustering

1. Let us consider a set of points in 2D space, and a circular sliding window centered at a point C (randomly selected) and having radius r as the kernel. Mean shift is a hill-climbing algorithm that involves shifting this kernel iteratively to a higher density region on each step until convergence.

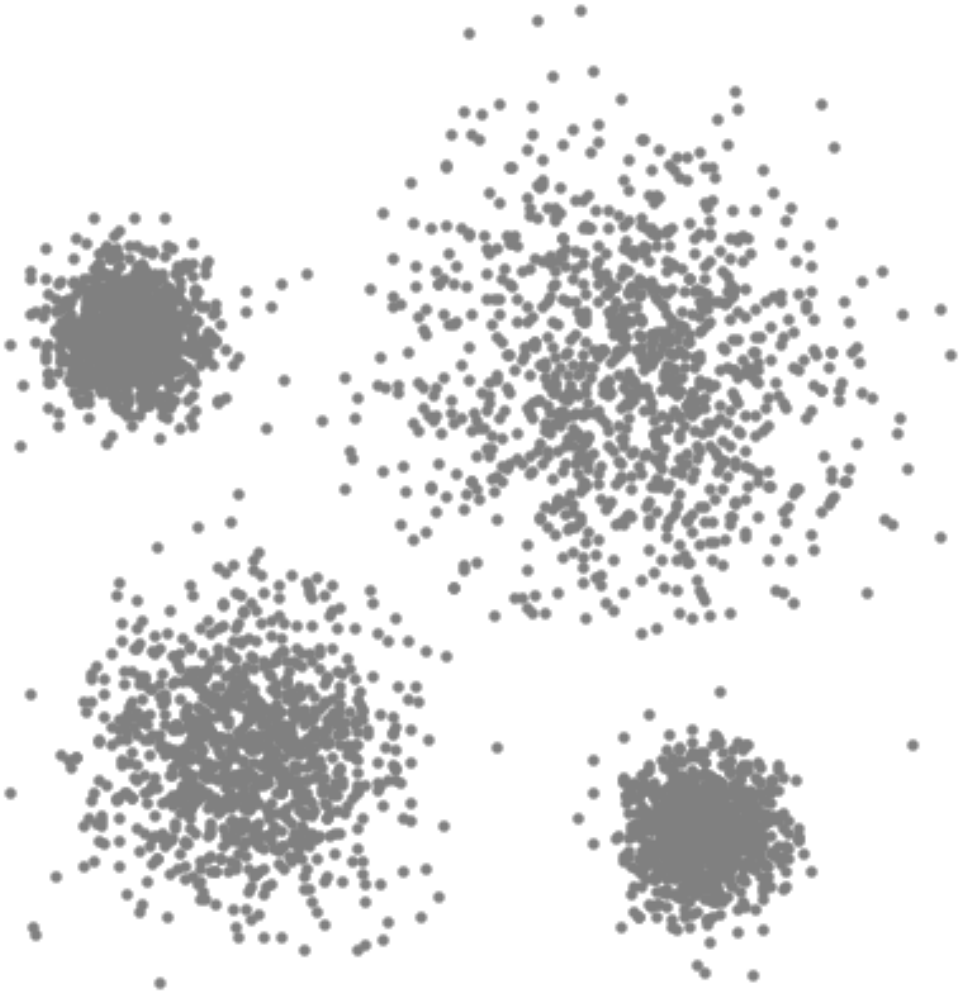
2. At every iteration, the sliding window is shifted towards regions of higher density by shifting the center point to the mean of the points within the window. The density within the sliding window is proportional to the number of points inside it. Naturally, it will gradually move towards areas of higher point density.

Unsupervised Learning
Mean-shift clustering

3. We continue shifting the sliding window according to the mean until there is no direction at which a shift can accommodate more points inside the kernel. We keep moving the circle until we no longer are increasing the density (i.e number of points in the window).

4. This process of steps 1 to 3 is done with many sliding windows until all points lie within a window. When multiple sliding windows overlap the window containing the most points is preserved. The data points are then clustered according to the sliding window in which they reside.

Unsupervised Learning
Mean-shift clustering



Fabrice LAURI
Maître de conférences à l'UTBM

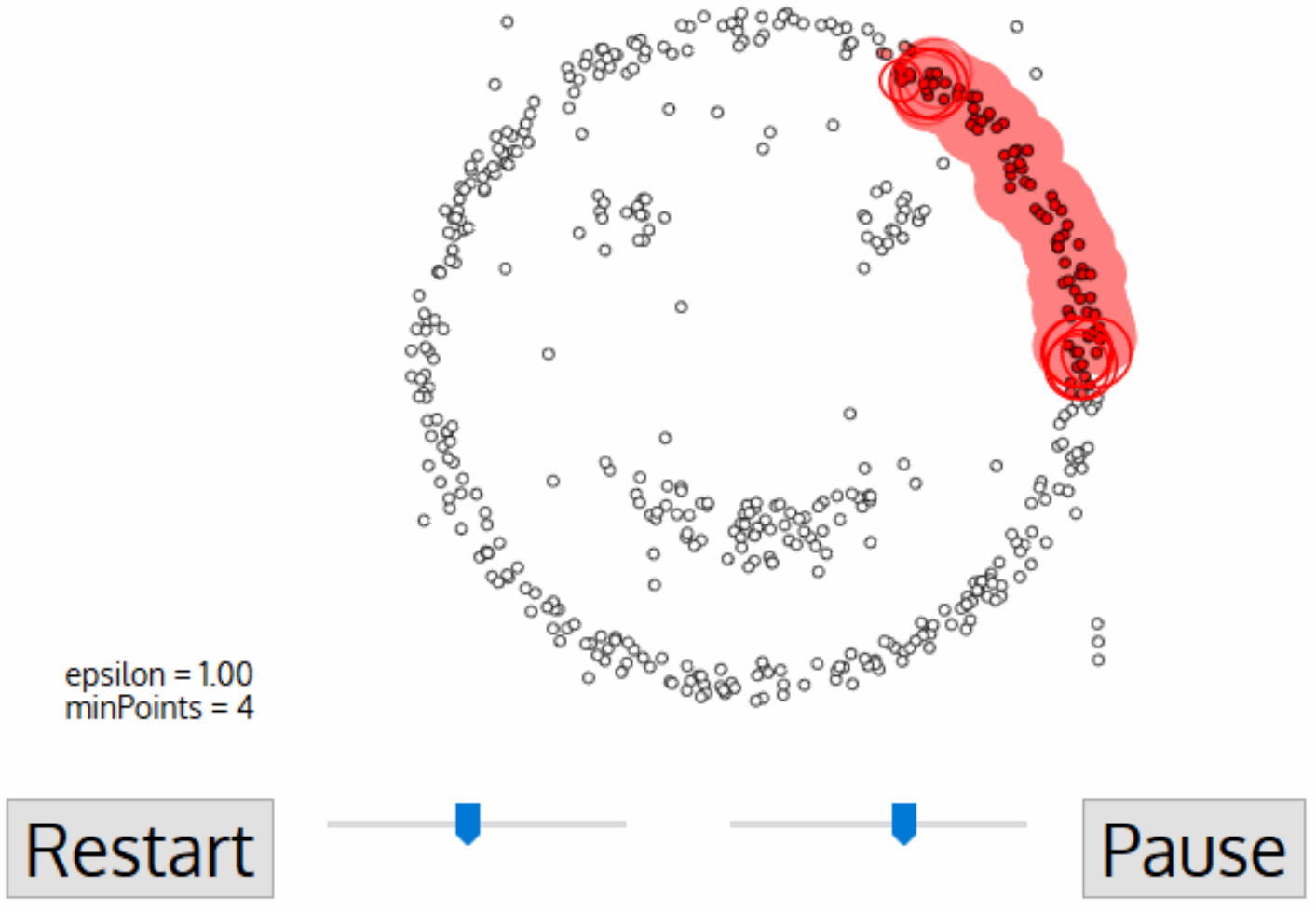
Unsupervised Learning
Mean-shift clustering drawbacks

In contrast to K-means clustering, there is no need to select the number of clusters as mean-shift automatically discovers this. That's a massive advantage.

The fact that the cluster centers converge towards the points of maximum density is also quite desirable as it is quite intuitive to understand and fits well in a naturally data-driven sense.

The drawback is that the choice of the window size/radius "r" can be non-trivial.

Unsupervised Learning
Density Based Spatial Clustering of Applications with Noise (DBScan)



Unsupervised Learning
DBScan

1. DBSCAN begins with an arbitrary starting data point that has not been visited. The neighborhood of this point is extracted using a distance epsilon ε (All points which are within the ε distance are neighborhood points).

2. If there are a sufficient number of points (according to minPoints) within this neighborhood then the clustering process starts and the current data point becomes the first point in the new cluster. Otherwise, the point will be labeled as noise (later this noisy point might become the part of the cluster). In both cases that point is marked as “visited”.

3. For this first point in the new cluster, the points within its ε distance neighborhood also become part of the same cluster. This procedure of making all points in the ε neighborhood belong to the same cluster is then repeated for all of the new points that have been just added to the cluster group.

Unsupervised Learning
DBScan

4. This process of steps 2 and 3 is repeated until all points in the cluster are determined i.e all points within the ε neighborhood of the cluster have been visited and labeled.

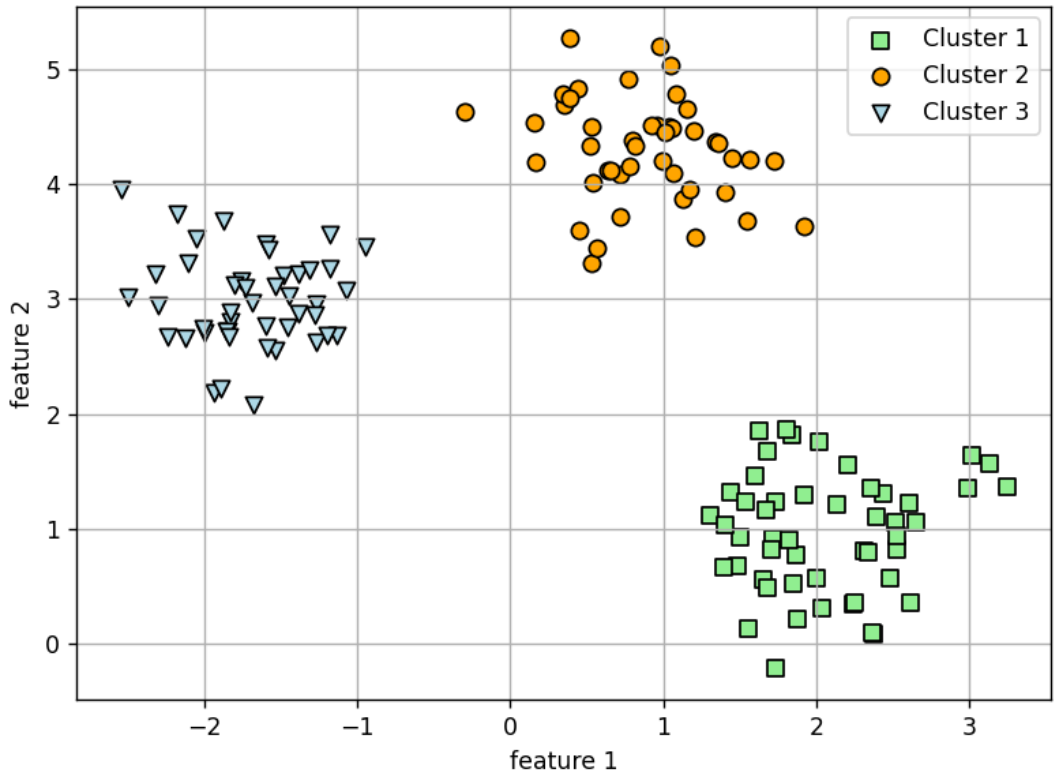
5. Once we're done with the current cluster, a new unvisited point is retrieved and processed, leading to the discovery of a further cluster or noise. This process repeats until all points are marked as visited. Since at the end of this all points have been visited, each point will have been marked as either belonging to a cluster or being noise.

Unsupervised Learning
DBScan advantages & drawbacks

DBSCAN poses some great advantages over other clustering algorithms. Firstly, it does not require a pre-set number of clusters at all. It also identifies outliers as noises, unlike mean-shift which simply throws them into a cluster even if the data point is very different. Additionally, it can find arbitrarily sized and arbitrarily shaped clusters quite well.

The main drawback of DBSCAN is that it doesn't perform as well as others when the clusters are of varying density. This is because the setting of the distance threshold ε and minPoints for identifying the neighborhood points will vary from cluster to cluster when the density varies. This drawback also occurs with very high-dimensional data since again the distance threshold ε becomes challenging to estimate.

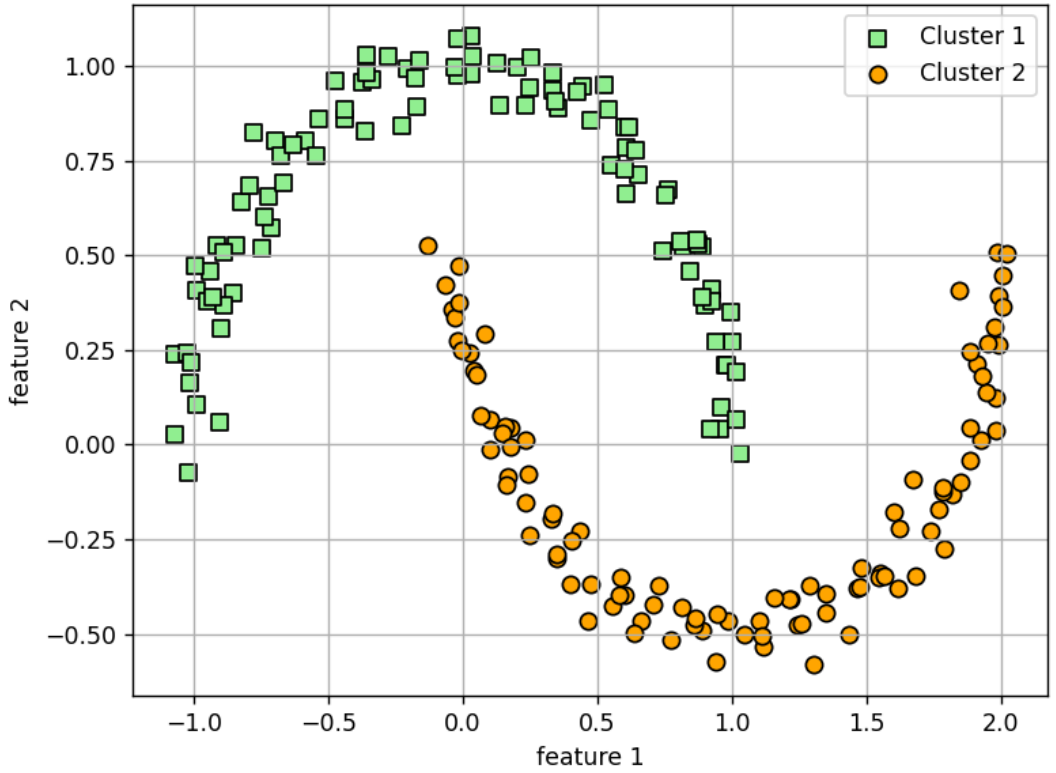
DBScan: Code snippet and results




```
from sklearn.cluster import DBSCAN

db=DBSCAN(
    eps=0.5,
    min_samples=5,
    metric='euclidean')
y_db=db.fit_predict(X)
```

Fabrice LAURI
Maître de conférences à l'UTBM

DBScan: Code snippet and results




```python
from sklearn.cluster import DBSCAN

db=DBSCAN(
    eps=0.2,
    min_samples=5,
    metric='euclidean')
y_db=db.fit_predict(X)
```

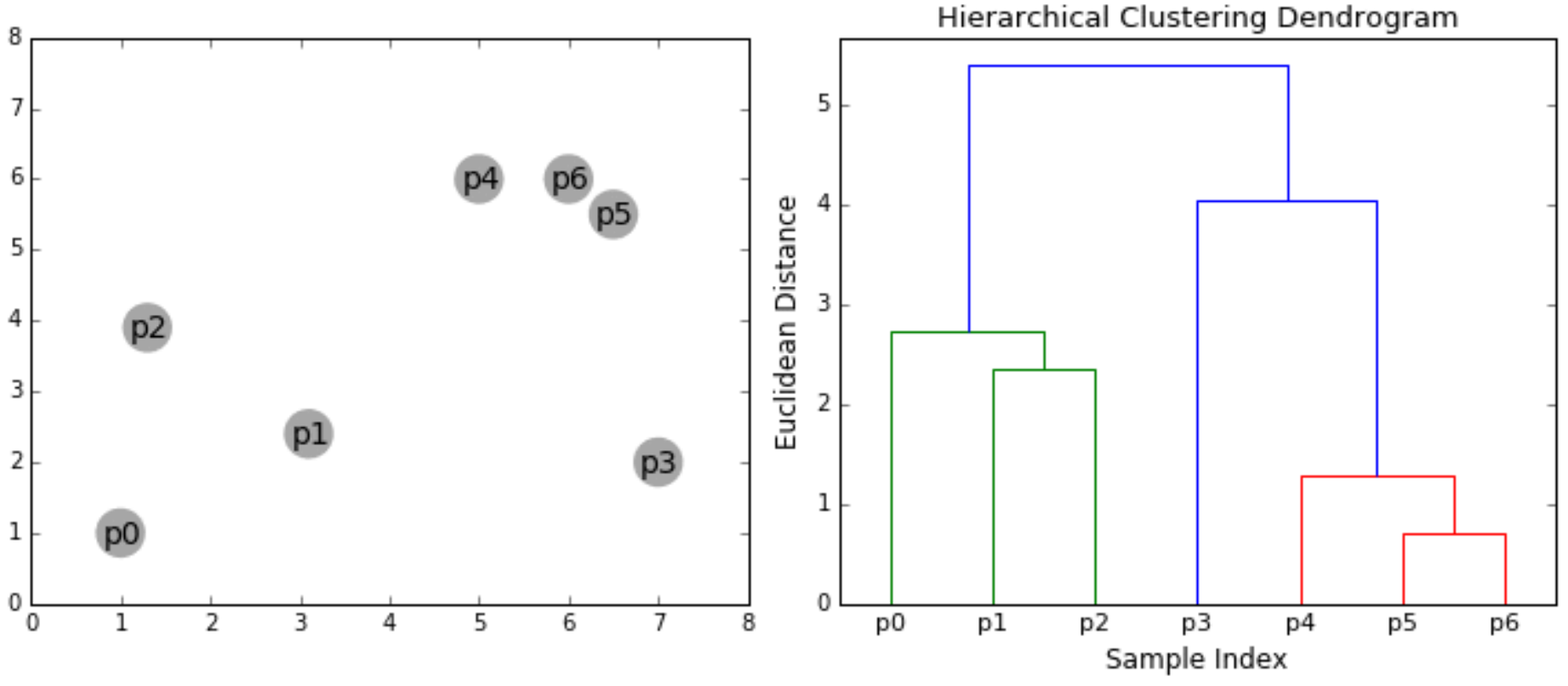
Fabrice LAURI
Maître de conférences à l'UTBM

Unsupervised Learning
Agglomerative Hierarchical Clustering

Hierarchical clustering algorithms fall into 2 categories: top-down or bottom-up.
Bottom-up algorithms treat each data point as a single cluster at the outset and then successively merge (or *agglomerate*) pairs of clusters until all clusters have been merged into a single cluster that contains all data points. Bottom-up hierarchical clustering is therefore called *hierarchical agglomerative clustering* or *HAC*.

Unsupervised Learning
Agglomerative Hierarchical Clustering

This hierarchy of clusters is represented as a tree (or dendrogram). The root of the tree is the unique cluster that gathers all the samples, the leaves being the clusters with only one sample.

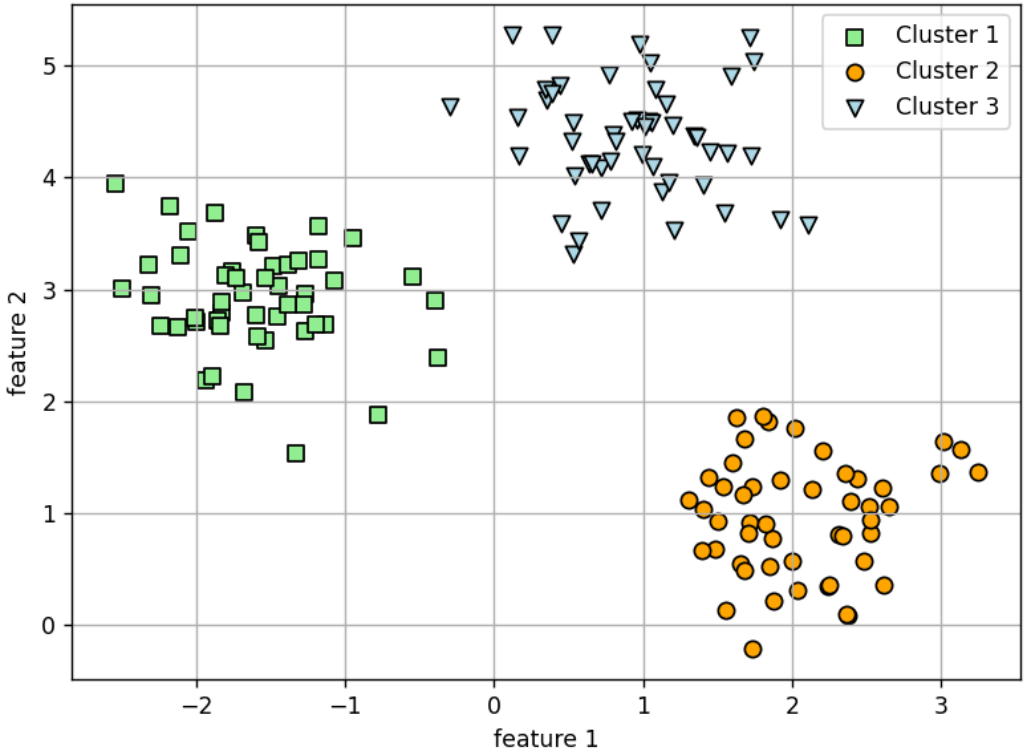


Fabrice LAURI
Maître de conférences à l'UTBM

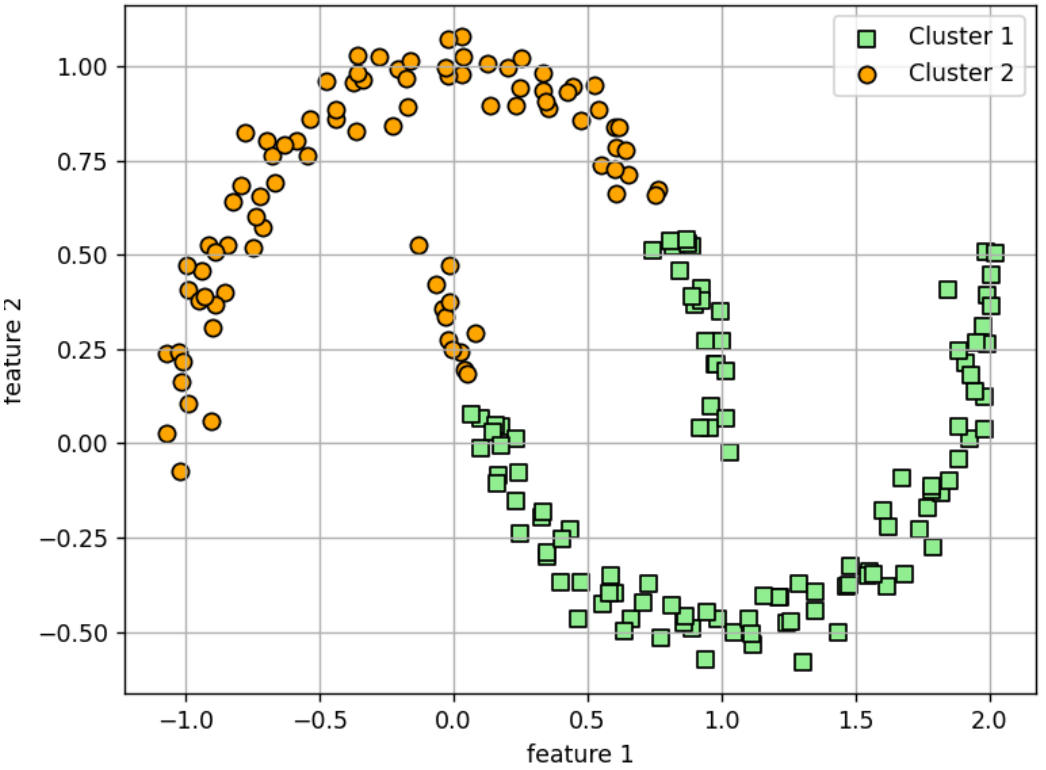
Unsupervised Learning
Agglomerative Hierarchical Clustering

1. We begin by treating each data point as a single cluster i.e if there are X data points in our dataset then we have X clusters. We then select a distance metric that measures the distance between two clusters. As an example, we will use *average linkage* which defines the distance between two clusters to be the average distance between data points in the first cluster and data points in the second cluster.
2. On each iteration, we combine two clusters into one. The two clusters to be combined are selected as those with the smallest average linkage. I.e according to our selected distance metric, these two clusters have the smallest distance between each other and therefore are the most similar and should be combined.
3. Step 2 is repeated until we reach the root of the tree i.e we only have one cluster which contains all data points. In this way we can select how many clusters we want in the end, simply by choosing when to stop combining the clusters i.e when we stop building the tree!

AHC: Code snippet and results




```
from sklearn.cluster import
AgglomerativeClustering

ahc=AgglomerativeClustering(
    n_clusters=3,
    metric='euclidean',
    linkage='complete')
y_ahc=ahc.fit_predict(X)
```




```
from sklearn.cluster import
AgglomerativeClustering

ahc=AgglomerativeClustering(
    n_clusters=2,
    metric='euclidean',
    linkage='complete')
y_ahc=ahc.fit_predict(X)
```

Unsupervised Learning
Agglomerative Hierarchical Clustering
Advantages and drawbacks

Hierarchical clustering does not require us to specify the number of clusters and we can even select which number of clusters looks best since we are building a tree. Additionally, the algorithm is not sensitive to the choice of distance metric; all of them tend to work equally well whereas with other clustering algorithms, the choice of distance metric is critical. A particularly good use case of hierarchical clustering methods is when the underlying data has a hierarchical structure and you want to recover the hierarchy; other clustering algorithms can't do this.

These advantages of hierarchical clustering come at the cost of lower efficiency, as it has a time complexity of $O(n^3)$, compared to the linear complexity of K-Means…

Unsupervised Learning
Top techniques results overview